Análise de Algoritmos de Busca aplicados ao Problema das Luzes (*Lights Out*)

Otávio de Oliveira Batista
Universidade Tuiuti do Paraná
Bacharelado em Ciência da Computação
otavio.batista@utp.edu.br

2026

Resumo

Este trabalho compara cinco algoritmos de busca aplicados ao quebra-cabeça *Lights Out*: BFS, DFS, Gulosa, A* e Hill Climbing com reinícios aleatórios. O tabuleiro começa sempre totalmente apagado e o objetivo é ligar todas as luzes. Variamos só o tamanho (2×2, 3×3, 5×5 e 7×7). Os algoritmos determinísticos rodam uma vez; o Hill Climbing roda várias vezes com sementes diferentes para medir taxa de sucesso. Os resultados mostram o compromisso clássico entre qualidade da solução, tempo e memória.

1 Introdução

O *Lights Out* é um quebra-cabeça em grid $N \times N$: cada clique inverte o estado da célula e de seus quatro vizinhos ortogonais. Começamos com tudo apagado e queremos ligar todas as luzes com o menor número de cliques. As regras são simples, mas o espaço cresce como 2^{N^2}, tornando o problema ideal para comparar busca cega, informada e local.

2 Fundamentação Teórica

2.1 Tipos de busca

Algoritmos de busca se dividem em três famílias:

- **Busca cega:** usa só a definição do problema. BFS e DFS.
- **Busca informada:** usa uma heurística $h(n)$ que estima quanto falta ao objetivo. Gulosa e A*; o A* combina $h(n)$ com o custo já percorrido $g(n)$.

- **Busca local:** trabalha em um único estado e tenta melhorá-lo passo a passo. Hill Climbing.

2.2 Métricas de comparação

- **Completude:** o algoritmo sempre acha solução quando ela existe?
- **Optimalidade:** a solução encontrada é a melhor possível?
- **Tempo:** quanto ele demora?
- **Espaço:** quanta memória ele usa?

2.3 Algoritmos implementados

- **BFS:** fila FIFO, nível por nível. Completa e ótima, mas gasta muita memória.
- **DFS:** pilha LIFO, vai fundo em um caminho. Menos memória, não garante a melhor solução. Limite de profundidade 15.
- **Gulosa:** usa só $h(n)$ = número de luzes apagadas. Rápida, mas não ótima.
- **A*:** usa $f(n) = g(n) + h(n)$. Adotamos $h(n) = \max(h_1, h_2)$, com $h_1(n) = \lceil a/5 \rceil$ (a = luzes apagadas; cada clique muda no máximo 5 células) e $h_2(n)$ = tamanho do maior grupo de apagadas mutuamente a distância de Manhattan $d_{\mathrm{M}}((r_1, c_1), (r_2, c_2)) = |r_1 - r_2| + |c_1 - c_2|$ maior que 2. Se $d_{\mathrm{M}} > 2$, as cruzes (célula + quatro vizinhos) não se sobrepõem e cada apagada exige um clique próprio. Ambos os limites nunca superestimam o custo real, logo o máximo também não, e o A* mantém a optimalidade. Usar o máximo é sempre melhor ou igual a usar cada limite isolado.
- **Hill Climbing com reinícios:** a cada passo, vai para o vizinho com melhor h. Se estaciona em um ótimo local, o tabuleiro é perturbado com cliques aleatórios e a subida recomeça (até 30 reinícios de 500 passos).

3 Metodologia

Os algoritmos estão em Python 3, um arquivo `.py` por algoritmo, compartilhando `tabuleiro.py` (operações básicas e auxiliar `medir()`, que mede tempo e memória de pico). O script `main.py` roda todos os experimentos. Ambiente: notebook Intel Core i5-10210U @ 1,60 GHz, 16 GB de RAM, Windows 11 Pro, Python 3.13.12, só biblioteca padrão.

Para cada execução registramos: taxa de sucesso, número de cliques, nós expandidos, tempo (`time.time()`) e memória de pico (`tracemalloc`). O estado inicial é sempre o tabuleiro totalmente apagado, evitamos gerar aleatórios porque nem toda configuração do

Lights Out tem solução (para $N = 5$, só cerca de 1/4 têm). BFS, DFS, Gulosa e A* são determinísticos e rodam uma vez; o Hill Climbing roda 10 vezes ($N \leq 5$) ou 5 vezes ($N = 7$) com sementes distintas.

Critérios de corte. O `main.py` aplica dois limites automáticos a toda execução: **tempo máximo** 600 s e **memória máxima** 4096 MB. A cada 500 iterações do laço principal, o algoritmo chama `verificar_corte()`; se algum limite é ultrapassado, a execução é interrompida (via exceção `CorteExcedido`) e o resultado é registrado como corte. Assim, todos os algoritmos são testados em todos os tamanhos de forma uniforme.

4 Resultados

No 2×2, todos convergem em poucos milissegundos e acham a solução de 4 cliques. A Tabela 1 mostra o 3×3: BFS e A* acham o ótimo (5 cliques); DFS e Gulosa acham, mas com 11. O Hill Climbing teve 60% de sucesso com caminhos de 1–3 cliques, valor aparentemente baixo, mas enganoso, pois não inclui os cliques aleatórios feitos nos reinícios.

Algoritmo	Taxa suc.	Cliques	Nós exp.	Tempo (s)	Mem. (KB)
BFS	100%	5	158	0,032	125,70
DFS	100%	11	263	0,024	105,46
Gulosa	100%	11	114	0,030	174,96
A*	100%	5	163	0,062	186,90
Hill Climbing	60%	1–3 (m. 2,0)	–	0,001–0,019	$\approx$ 4,5

Tabela 1: Resultados no tabuleiro 3×3 apagado (HC: 10 execuções).

A Tabela 2 mostra 5×5 e 7×7 com os cortes ativos. No 5×5, só a Gulosa termina (25 cliques, 0,8 s, 8,6 MB); BFS e A* são cortados por tempo com picos de memória bem abaixo do limite, mostrando que o gargalo aqui é tempo. A DFS esgota a busca em 568 s com 3 milhões de nós sem achar solução, o limite de profundidade 15 é pequeno demais. No 7×7, BFS e DFS seguem cortadas por tempo; a Gulosa passa a ser cortada por memória (pico 4,20 GB); o A*, graças à heurística mais forte, mantém-se em 1,40 GB e é cortado por tempo. O Hill Climbing tem 0% de sucesso em ambos os tamanhos.

Instância	Algoritmo	Status	Cliques	Nós/passos	Tempo (s)	Mem. pico
5×5	BFS	CORTE (tempo)	–	–	620	$\approx$ 1,15 GB
5×5	DFS	Não achou	–	$3{,}0 \times 10^6$	568	$\approx$ 1,56 GB
5×5	Gulosa	Sim	25	466	0,79	$\approx$ 8,65 MB
5×5	A*	CORTE (tempo)	–	–	608	$\approx$ 1,12 GB
5×5	Hill Climbing	Não achou	–	101–106	0,36–1,18	$\approx$ 15 KB
7×7	BFS	CORTE (tempo)	–	–	621	$\approx$ 830 MB
7×7	DFS	CORTE (tempo)	–	–	618	$\approx$ 1,67 GB
7×7	Gulosa	CORTE (memória)	–	–	658	$\approx$ 4,20 GB
7×7	A*	CORTE (tempo)	–	–	613	$\approx$ 1,40 GB
7×7	Hill Climbing	Não achou	–	$\approx$ 212	0,47–0,65	$\approx$ 42 KB

Tabela 2: Resultados em tabuleiros maiores, com cortes de 600 s e 4 GB (HC: 10 execuções).

5 Discussão

Os resultados mostram o compromisso clássico entre achar a melhor solução, ser rápido e economizar memória. A BFS garante o ótimo, mas já no 3×3 gasta mais memória que a DFS, e em $N \geq 5$ não termina. A DFS economiza memória mas retorna soluções mais longas (11 contra 5 cliques no 3×3) e depende fortemente do limite de profundidade. A Gulosa escala melhor que as cegas (resolveu o 5×5 em 0,8 s), mas a qualidade oscila, retornou 11 cliques no 3×3, mais que o dobro do ótimo, porque ignora o custo acumulado. No 7×7, sem $g(n)$ que controle a *fringe*, ela estoura a memória.

O A* foi o mais equilibrado em tabuleiros pequenos, e a heurística fez diferença ao escalar. A Tabela 3 compara a heurística inicial $h_1(n) = a/5$ com a combinada $h(n) = \max(\lceil a/5 \rceil, h_2)$. A combinada reduziu $\approx 30\%$ dos nós no 3×3 e $\approx 17\%$ do pico de memória no 5×5. No 7×7, mudou qualitativamente o gargalo: de memória (antes passava de 4 GB) para tempo, com memória caindo a cerca de um terço. Ainda assim, nem o 5×5 nem o 7×7 terminam em 10 minutos, para escalar de fato, seriam necessárias heurísticas algébricas sobre F_2 ou IDA*.

Instância	$h_1 = a/5$ (simples)	$h = \max(\lceil a/5 \rceil, h_2)$ (combinada)
3×3	233 nós; 156 KB; 0,06 s	163 nós; 187 KB; 0,06 s
5×5	CORTE tempo 662 s; pico 1,35 GB	CORTE tempo 608 s; pico 1,12 GB
7×7	CORTE **memória** 579 s; pico 4,20 GB	CORTE tempo 613 s; pico **1,40 GB**

Tabela 3: Heurística simples vs combinada no A*.

O Hill Climbing foi o mais leve, mas teve 0% de sucesso em 5×5 e 7×7: o espaço de busca, visto pela função "luzes apagadas", tem muitos ótimos locais dos quais a subida pura com reinícios não escapa. O Simulated Annealing talvez parecesse uma solução, mas cada clique muda até cinco células: movimentos aleatórios aceitos pela temperatura alta não são pequenas perturbações, e sim reconfigurações grandes que tendem a desorganizar o

tabuleiro em vez de orientá-lo ao objetivo.

6 Conclusão

Não há um algoritmo sempre melhor: a escolha depende do tamanho e do que se quer. Em $N \leq 3$, BFS e A* resolvem em milissegundos com optimalidade. Em tamanhos médios, o A* se destaca na teoria mas depende de uma boa heurística para escalar. A Gulosa foi a única busca informada que resolveu o 5×5 no experimento, ainda que com soluções não ótimas. O Hill Climbing é o mais leve em recursos mas teve 0% de sucesso em $N \geq 5$, evidenciando a presença de muitos ótimos locais. Como trabalhos futuros: heurísticas mais fortes via álgebra linear sobre F_2; IDA* para reduzir memória; e algoritmos algébricos (eliminação de Gauss em $GF(2)$) como baseline polinomial.

Referências

Marlow Anderson and Todd Feil. Turning lights out with linear algebra. *Mathematics Magazine*, 71(4):300–303, 1998. doi: 10.1080/0025570X.1998.11996658.

Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. doi: 10.1109/TSSC.1968.300136.

Scott Kirkpatrick, C. Daniel Gelatt, and Mario P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983. doi: 10.1126/science.220.4598.671.

Matthew A. Madsen. Lights out: Solutions using linear algebra. Expository paper, 2008. Disponível em `http://cau.ac.kr/~mhhgtx/courses/LinearAlgebra/references/MadsenLightsOut.pdf`.

Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, Hoboken, NJ, 4 edition, 2021. ISBN 9780134610993.